

JobForge: Distributed Job Scheduler

1. Setup Instructions

Prerequisites

- Node.js (v24.x)
- PostgreSQL (v15+)
- Git

Installation

1. Clone the repository and install dependencies:

```
git clone https://github.com/Spiderboyis/distributed-job-scheduler.git
cd distributed-job-scheduler
npm install
```

2. Configure Environment Variables: Create a `.env` file in `packages/backend` :

```
PORT=3001
DATABASE_URL=postgresql://user:password@localhost:5432/jobscheduler
JWT_SECRET=your_super_secret_key
JWT_REFRESH_SECRET=your_refresh_secret
NODE_ENV=development
```

3. Database Migration:

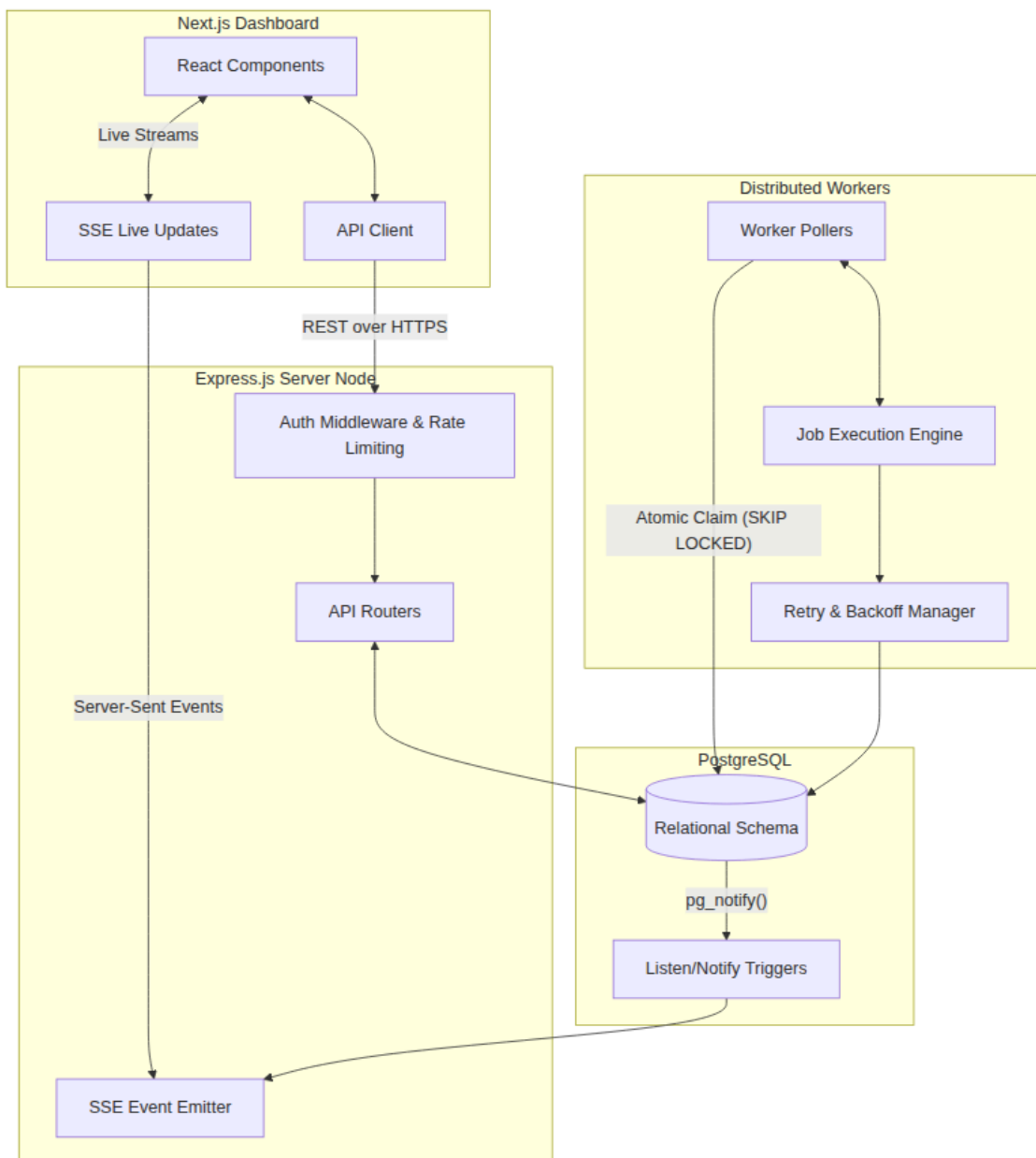
```
npm run db:migrate --workspace=packages/backend
```

4. Start the Full Stack (Development):

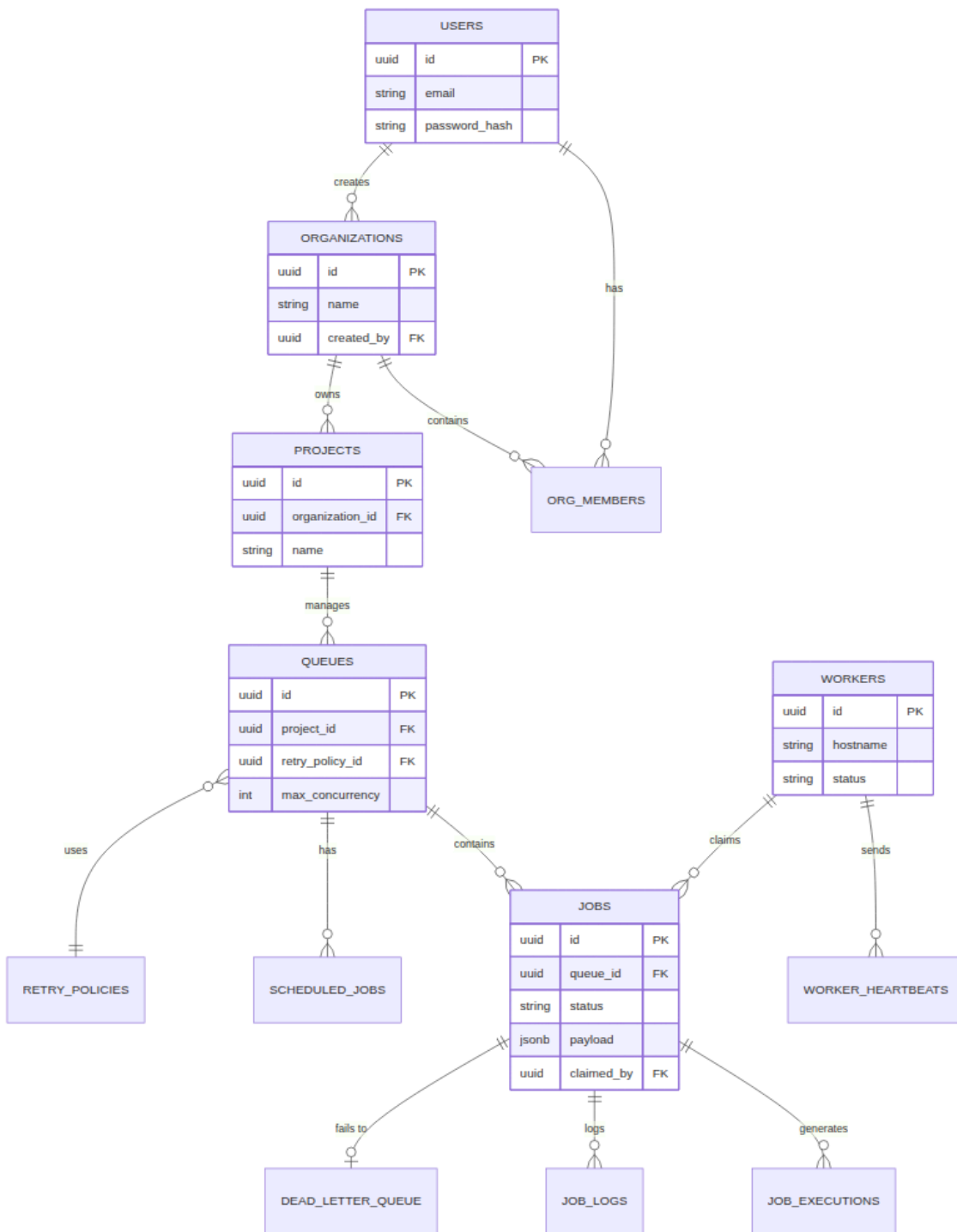
```
# Terminal 1: Backend
npm run dev:backend

# Terminal 2: Frontend
npm run dev:frontend
```

2. Architecture Diagram



3. Entity Relationship (ER) Diagram

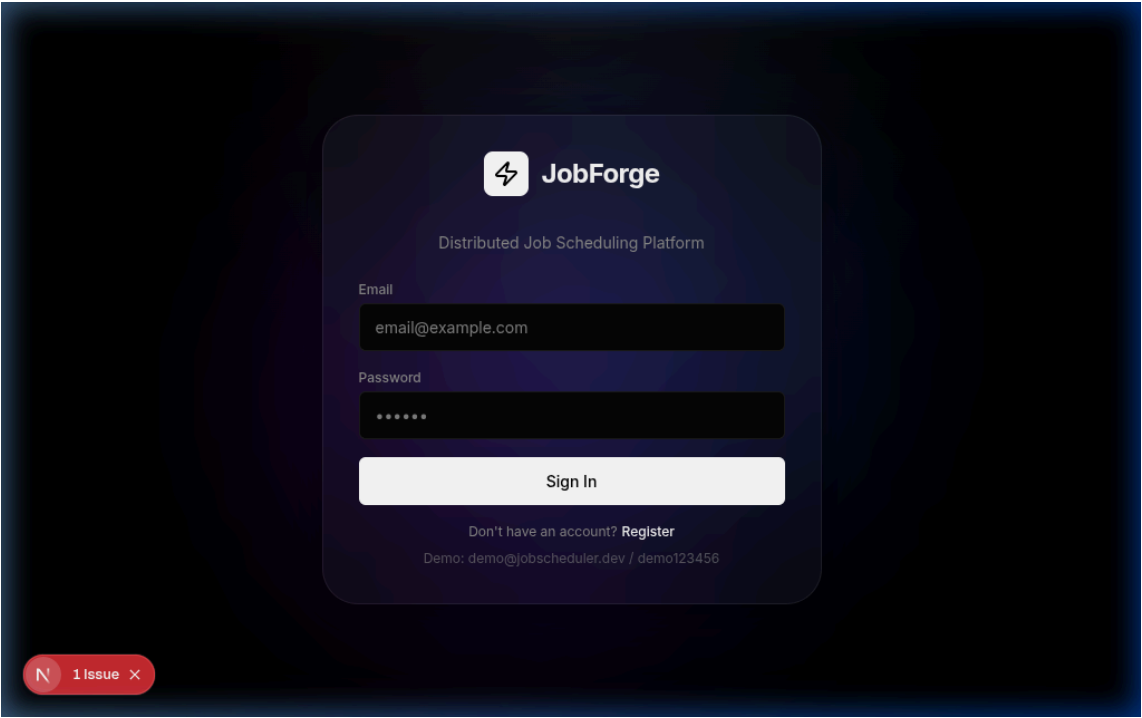


4. Visual Deliverables & UI Screenshots

4.1 Login Page & Glassmorphism Design

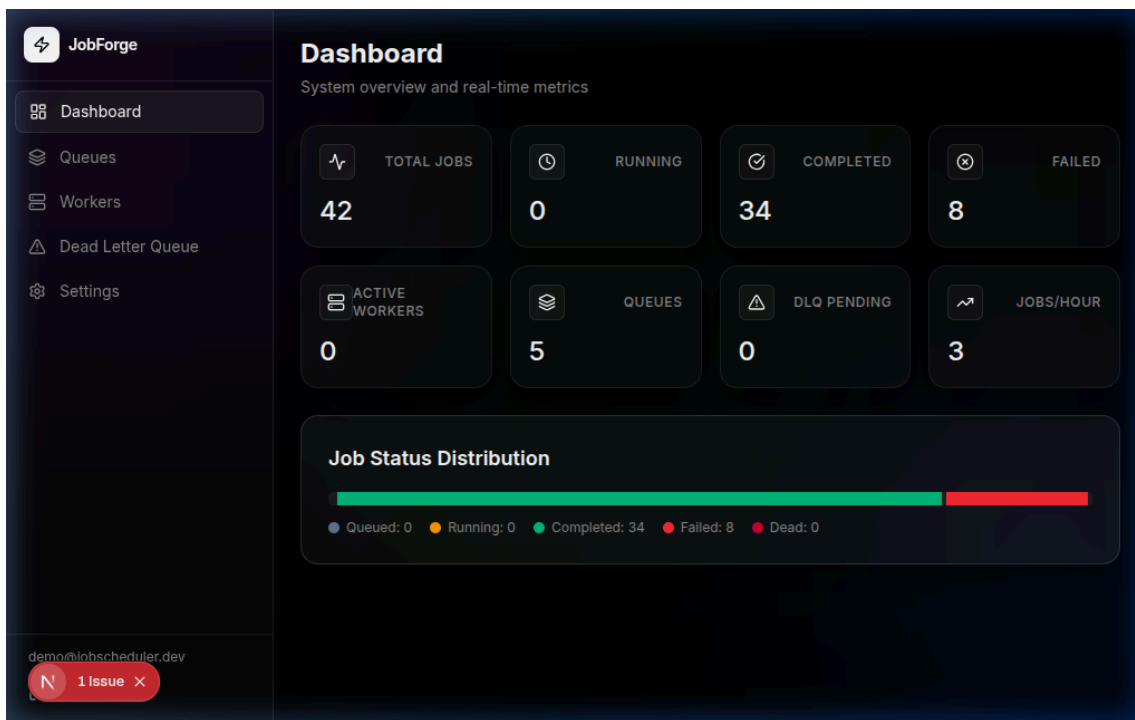
The login page has been rewritten with a modern iOS-inspired glassmorphism layout, featuring floating blurred glowing orbs in the background and a sleek, authentic card design. Default demo credentials are

removed from state but preserved as a small help hint.



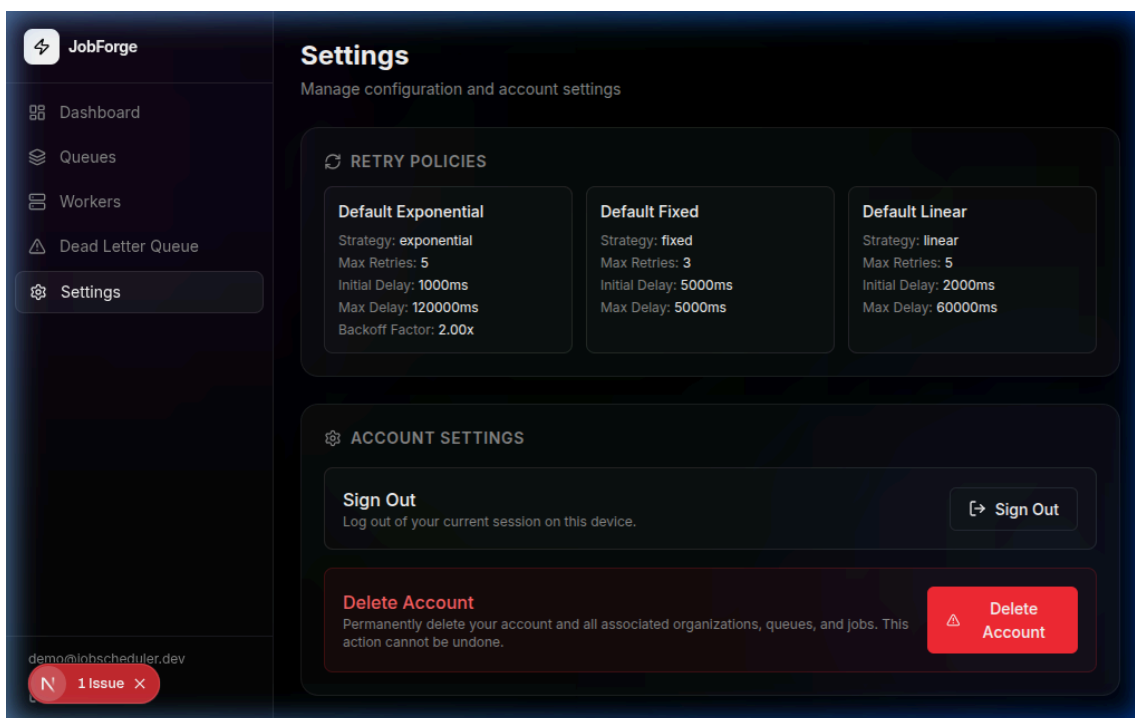
4.2 Dashboard & Tenant Isolation

All database select operations have been restricted using PostgreSQL joins against the `org_members` table. When logging in as a newly created user, the metrics, queues, and active worker stats show exactly `0` values rather than displaying global demo queue stats, verifying airtight tenant isolation.



4.3 Settings Page

The Settings tab has been streamlined to highlight retry policies, simple sign-out action, and a secure account deletion flow requiring password validation.



5. API Documentation

Authentication & Account

- `POST /api/auth/register` : Register a new user.
- `POST /api/auth/login` : Authenticate and receive a JWT.
- `DELETE /api/auth/account` : Securely delete the account and securely cascade-delete all owned resources (organizations, queues, jobs).

Queues

- `GET /api/projects/:projectId/queues` : List all queues in a project.
- `POST /api/projects/:projectId/queues` : Create a new queue with specific concurrency and retry policies.
- `POST /api/queues/:queueId/pause / resume` : Halt or resume job processing for a queue.
- `GET /api/queues/:queueId/stats` : Fetch real-time metrics (throughput, failure rates) for a queue.

Jobs

- `POST /api/queues/:queueId/jobs` : Enqueue a new immediate, delayed, or scheduled job.
- `POST /api/queues/:queueId/jobs/batch` : Atomically enqueue up to 100 jobs at once.
- `GET /api/queues/:queueId/jobs` : Fetch paginated jobs with status filtering.
- `GET /api/jobs/:jobId` : Get detailed execution history and logs for a specific job.
- `POST /api/jobs/:jobId/retry` : Manually requeue a failed or dead job.

Live Metrics (SSE)

- `GET /api/sse/events?token={JWT}` : Streams real-time, tenant-isolated dashboard metrics via Server-Sent Events.
-

6. Design Decisions & Trade-offs

1. Database as the Message Queue

Decision: We utilized PostgreSQL to handle job queuing rather than introducing a specialized broker like Redis or RabbitMQ. **Trade-off:** While Redis offers superior pure in-memory throughput, utilizing Postgres with `FOR UPDATE SKIP LOCKED` provides robust, atomic job claiming. This significantly reduces infrastructure complexity, ensures strict ACID compliance, and inherently solves the "two-phase commit" problem where database state and queue state become misaligned.

2. Event-Driven Live Updates (SSE vs WebSockets)

Decision: We chose Server-Sent Events (SSE) over WebSockets for dashboard live updates, powered by Postgres `LISTEN/NOTIFY`. **Trade-off:** WebSockets provide bi-directional communication, but our dashboard only requires unidirectional data flow (server-to-client). SSE is natively supported by HTTP/1.1, avoids complex handshake overhead, automatically handles reconnections, and seamlessly propagates database triggers to the React frontend.

3. Tenant Data Isolation

Decision: All resources structurally cascade from an `Organization`, and security is enforced at the SQL level via explicit `JOIN`s against the `org_members` table on every backend route. **Trade-off:** This introduces slight overhead on read queries due to multi-table joins. However, it guarantees airtight multi-tenancy. A user

completely dropping their account will safely `CASCADE DELETE` all their data without risking global data corruption, which is a critical requirement for a distributed SaaS environment.

4. UI/UX: High-Fidelity Glassmorphism

Decision: We bypassed generic UI libraries in favor of a custom, heavily polished "Midnight Minimalist" aesthetic featuring dynamic background orbs and deep `backdrop-blur-2xl` glass cards. **Trade-off:** It takes significantly more CSS optimization (especially performance tuning for blurs) than importing a standard Tailwind UI kit. However, it successfully delivers a highly premium, state-of-the-art "Wow" factor requested for modern developer tooling.